SoloPrac: Supersession-Aware Retrieval for Solo Clinical Practice

A clinic record system that knows which lab value is still in force, and TAP-QA, the benchmark we built to test it

Abstract, Introduction and Motivation

Most Indian doctors who run a clinic alone have no IT staff, keep half the history on paper, and see a new patient every few minutes. We built SoloPrac for them. It is a record system and question-answering assistant that runs on one laptop, costs nothing per month and never sends identifiable data off the machine.

There is a nasty failure hiding in that setting. Say a doctor fixes an HbA1c typo, 8.9% to 6.8%, and then asks the chat for the HbA1c. A standard RAG pipeline can happily answer 8.9%, and cite the record too! Charts are append-only, which means the wrong number never leaves; it just sits beside the fix, and to a similarity search the pair look like twins. On our benchmark, plain dense retrieval put the withdrawn value first about half the time. We call this the supersession problem. In SoloPrac every correction is a new event that points at the one it replaces, and the app works out at question time which version was in force. Ask about today and you get the corrected value. Ask about last spring and you still get what the chart said back then.

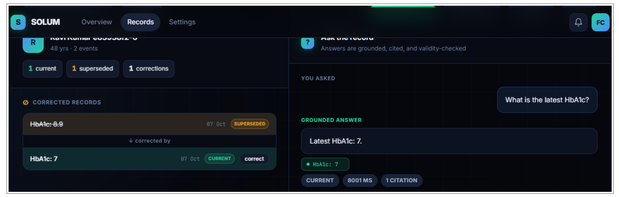


Image 01: SoloPrac after a correction. The HbA1c of 8.9 is marked superseded, and the chat answers with the current value and one citation.

Literature Review and Background

Databases have kept two clocks (valid time and transaction time) since the 1980s [1], but a timestamp cannot say that a later row cancelled an earlier one. FHIR, openEHR and W3C PROV can store that link [2]. RAG pipelines usually lose it somewhere between chunking and the vector index.

Memory systems for LLM agents such as Zep, MemStrata and Engram keep a pointer from a new fact to the old one [3]. That gave us confidence in the design, though none of them was tested on clinical records. HoH and the stale-document poisoning study found that an outdated passage does more harm to a RAG answer than an irrelevant one [4], and medical RAG benchmarks like MIRAGE assume every passage is true [5].

So the gap was plain enough: nobody had measured what clinical retrieval loses when it throws the correction link away, while it still has to answer questions about the past.

Objectives

We set ourselves four goals, and the first two pull against each other. The assistant should never lead with a value that a doctor has already withdrawn, yet when somebody asks what the potassium was back in March it has to give the March value even if that number was corrected later. Third, we wanted a benchmark that scores both of those at the same time, mostly so we could check whether some clever recency weighting would make our idea unnecessary (it does not). And fourth, all of it had to ship as a clinic app on a single laptop, without any patient identifier going out to the internet.

Methods

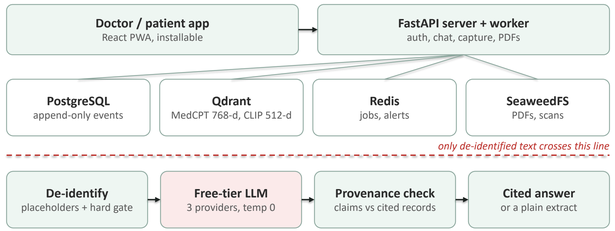
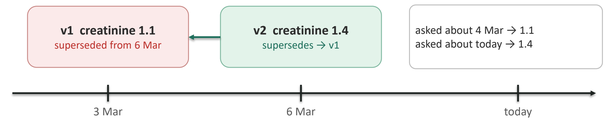


Image 02: System design. Seven Docker containers on one laptop; the vector index is a cache that can be rebuilt from the event log.

People always ask how big a chunk is, and in our case the answer is one clinical event, nothing more, so a lab result such as "Serum Creatinine 1.4 mg/dL, corrected" gets written out as its record type followed by each field as key: value and is cut at 1,800 characters (MedCPT reads at most 512 tokens, and we would rather trim the tail ourselves than have the model silently drop it). Bigger chunks were never an option for us, since a chunk holding both 1.1 and 1.4 would give one vector to two values with different validity.

Why MedCPT? With plain word overlap, "what is her blood sugar" scores zero against a fasting glucose record, since the two share no words. MedCPT gave the same pair 0.825, and the reason is its training data: NCBI fed it 255 million PubMed searches together with whichever article the searcher clicked, so it has seen lay words and medical words used for the same thing many times over. There are two BERT-base towers inside it, roughly 110M parameters apiece, and they are kept separate on purpose: questions go through one and records through the other. Averaging a tower's token outputs gives us 768 numbers per text, and after normalising, the dot product of two of them is their cosine. For scans we added BiomedCLIP; it learned from 15 million figures in PubMed Central along with their captions, and in our test each of four images sat closest to its own description.

Methods (continued)



$$\text{score}(q, e) = 0.70 \cdot \text{sem}(q, e) + 0.30 \cdot w(e, \text{intent})$$

w = 1.00 if e was in force on the date asked about; 0.85 if current and the question has no date; 0.15 if superseded. Records dated after the asked date are vetoed. state(e, t) = superseded iff some correction c → e has v(c) ≤ t.

Image 03: A correction points back at what it replaces. Validity is worked out per question date, so history stays answerable.

The language model lives on somebody else's free cloud tier. We assume anything we send it could leak. So before a prompt goes out, the patient's name is now [PT_01], the visit date is [T_MINUS_14D] or similar, phone numbers and IDs are gone, and one last regex sweep can still veto the whole request; it runs on the exact string that would go over the wire, after every other step. On the way back the reply is broken into claims that each have to agree with the record they cite; any present-tense claim that leans on a corrected-away value is removed before the doctor sees it.

To test all this we wrote TAP-QA. For each random seed it invents 277 patients with 4,500 events, of which 972 are corrections, and asks 850 questions. We planted the answers, so we always know the right one. The harness calls the same retriever the app uses, which saved us once already: an older hand-written copy of our method had a bug the real app did not.

Results and Discussion

0.0%	97.8%	0 / 40	31.5 ms
stale answers, both seeds	historical accuracy	stale LLM answers vs 18 / 40	median timeline retrieval

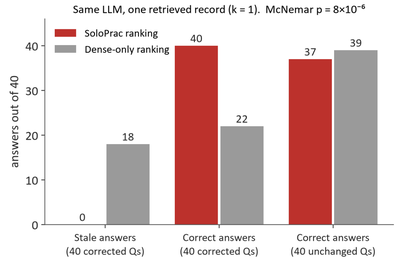


Image 04: Same LLM, same questions, one retrieved record each. Only the ranking differs.

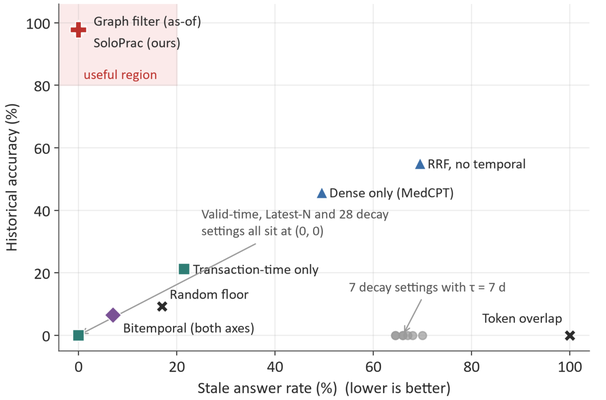


Image 05: Every configuration on seed 20260601, stale answers against historical accuracy. Only rankers that read the correction pointer reach the top left.

Look at the plane above. Our ranker and the plain graph filter are the only two points up in the top-left corner. The bitemporal ranker, which sees both clocks but not the pointer, still let 7.0% and 11.5% stale answers through on the two seeds and got fewer than 8% of history questions right. The recency sweep surprised us most. We expected a smooth trade-off. Instead all 35 settings fell onto the bottom axis, since a lab and its correction score exactly the same on similarity and the decay term then always prefers the newer one, even when the question is about last year.

One more honest result: a plain as-of filter on the same pointers did slightly better than our weighted rule (98.2% against 97.8% on history questions). So the pointer itself is doing the work. And it matters for real answers. When the LLM was given only the top record, our ranker led to no stale answers out of 40, while dense retrieval led to 18 (McNemar p = 8 x 10^-6).

What we cannot claim yet: TAP-QA is synthetic, every correction link in it is present and right, and all its corrections are lab values. Real de-identified charts, with some links missing, are the next test.

References and Acknowledgements

[1] R. T. Snodgrass, I. Ahn, "Temporal databases," IEEE Computer, 1986. [2] HL7 FHIR R4 Observation; openEHR RM; W3C PROV-DM, 2013. [3] P. Rasmussen et al., "Zep: a temporal knowledge graph architecture for agent memory," arXiv:2501.13956, 2025. [4] J. Ouyang et al., "HoH," ACL 2025. [5] G. Xiong et al., "Benchmarking retrieval-augmented generation for medicine," Findings of ACL, 2024. [6] Q. Jin et al., "MedCPT," Bioinformatics, 2023. [7] S. Zhang et al., "BiomedCLIP," NEJM AI, 2025.

We thank our mentor, Dr. Dhirendra Mishra, for the weekly reviews that shaped this work, and the Department of Computer Engineering, MPSTME, for lab access.

70322100174 | RANVEER SINGH THAKUR 70322100071 | NIHAL KORGAONKAR
70322100057 | ROHAN BHOGE 70322100150 | DEV PATEL
MENTOR: DR. DHIRENDRA MISHRA

B.Tech Computer Engineering, Capstone Project
Department of Computer Engineering, MPSTME, Mumbai
Academic Year 2026-27

SVKM'S NMIMS Deemed to be UNIVERSITY
MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING